

Optimisation de l'algorithme Apriori avec la moyenne tronquée

Daddi hammou mahdi

202039080304

G1 M1-MV

[Intorduction](#)

[Apriori avec `min\_sup` fixe](#)

[Apriori avec `min\_sup` dynamique](#)

[Code initial](#)

[Code final](#)

[Tests](#)

[Dataset](#)

[Préparation des données](#)

[Règles d'association](#)

[Conclusion](#)

Intorduction

L'algorithme Apriori est une méthode de fouille de données utilisée pour découvrir des associations entre différents éléments dans un ensemble de données. Cette approche est largement utilisée dans divers domaines, tels que le commerce électronique, la recommandation de produits et l'analyse de paniers d'achat.

Un problème courant avec l'algorithme Apriori réside dans la nécessité de spécifier un seuil de support minimal fixe. Ce seuil détermine le nombre minimum d'occurrences qu'un ensemble d'éléments doit avoir pour être considéré comme "fréquent". La fixation d'un seuil de support minimal peut être délicate, car elle nécessite une connaissance préalable des données et peut ne pas convenir à toutes les situations.

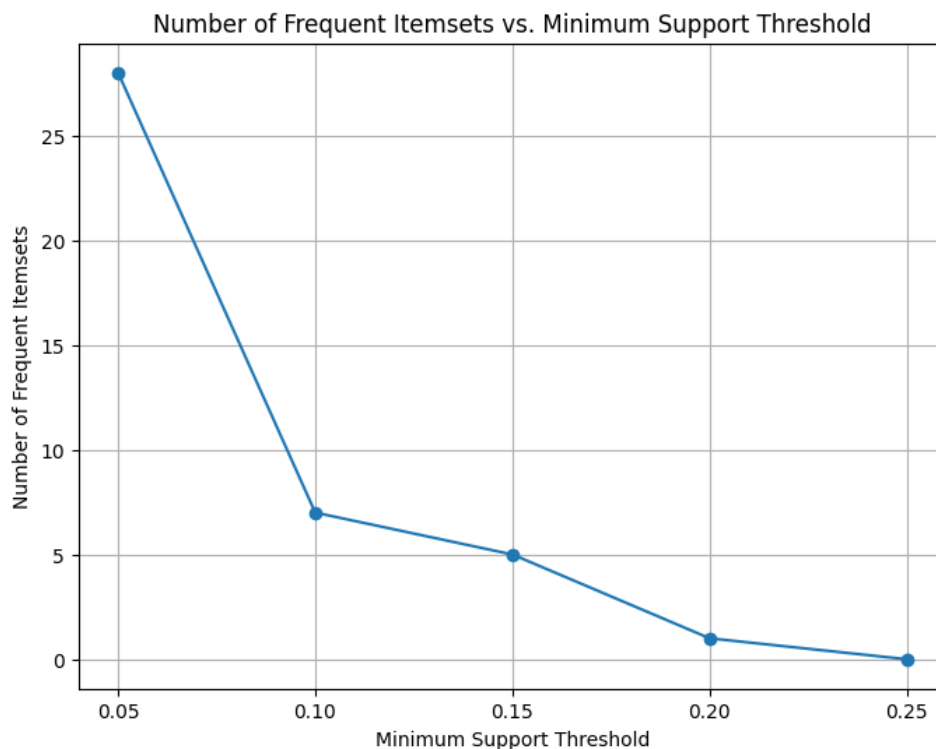
Dans notre étude, nous proposons une solution pour ce problème. nous examinons les avantages d'un seuil de support minimal dynamique et les

techniques permettant de l'ajuster de manière adaptative en fonction des caractéristiques des données.

Apriori avec `min_sup` fixe

Nous avons utilisé l'implémentation a priori de `mlexend` car elle utilise un support minimum fixe.

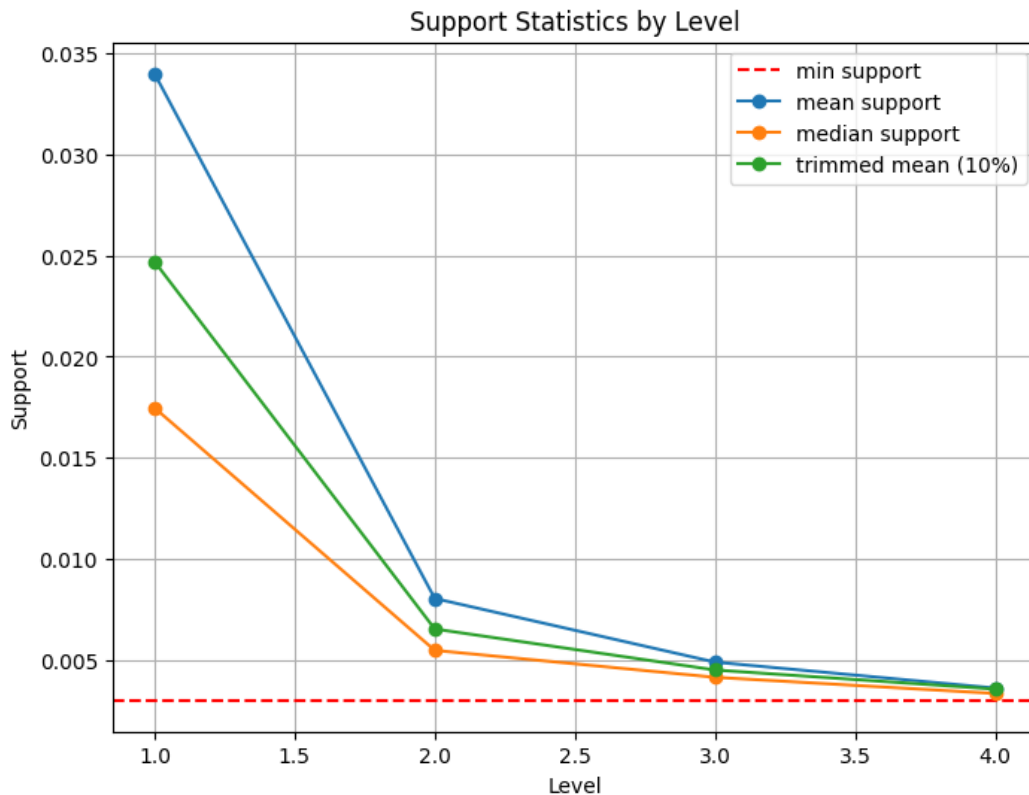
comme le graphique ci-dessous montre comment de petits changements dans `min_sup` modifient considérablement la sortie des ensembles d'éléments fréquents, ce qui rend presque impossible d'avoir un `min_sup` fixe pouvant être utilisé à tous les niveaux.



pour notre prochaine expérience, nous avons fixé un `min_sup` de 0,003 pour illustrer comment le support moyen, median, et la moyenne tronquée (10%) évolue à mesure que le niveau augmente.

$$\text{Mean} = \frac{1}{n} \sum_{i=1}^n x_i$$

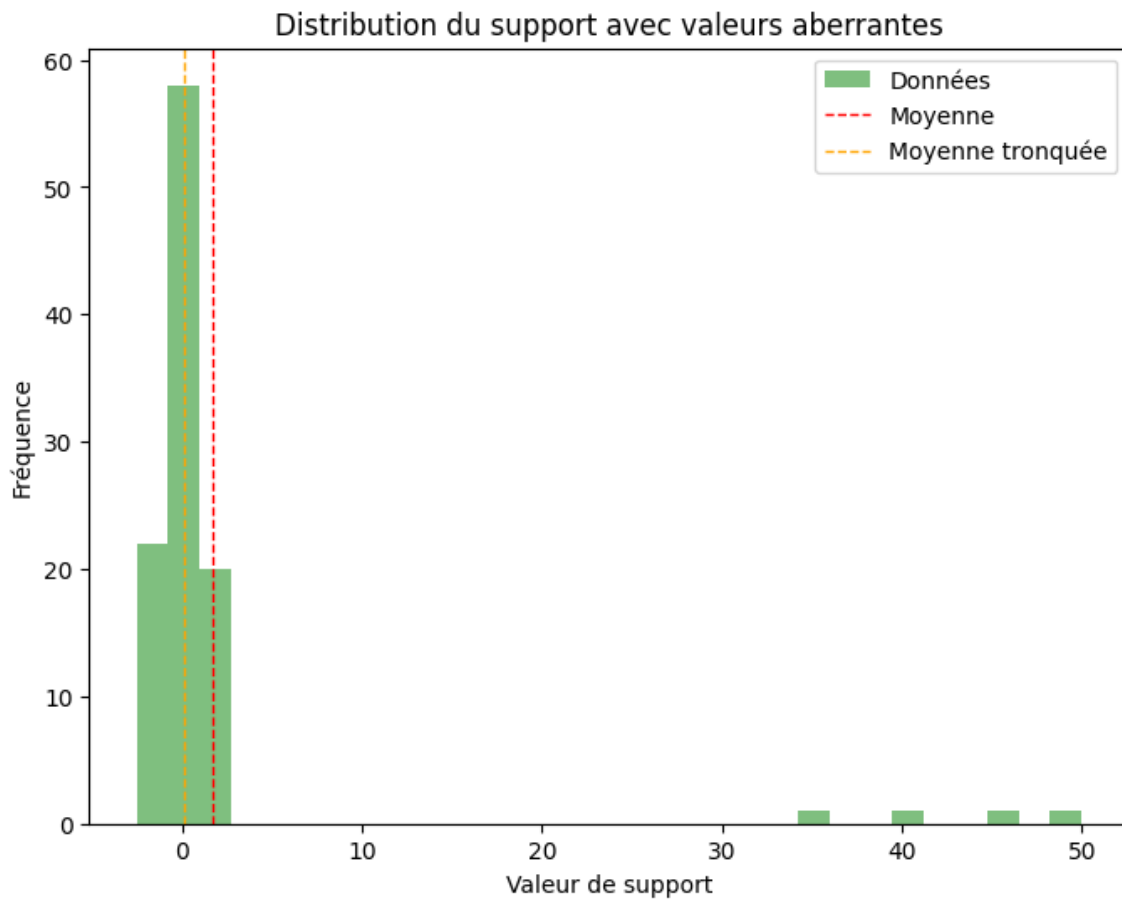
$$\text{Trimmed Mean} = \frac{1}{n - 2k} \sum_{i=k+1}^{n-k} x_{(i)}$$



1. **Moyenne du support** : La moyenne du support a tendance à être plus élevée que les autres mesures. Cela est attendu car la moyenne est sensible aux valeurs extrêmes.
2. **Moyenne tronquée** : La moyenne tronquée, qui supprime un certain pourcentage de valeurs extrêmes des deux extrémités de la distribution avant de calculer la moyenne, a tendance à être inférieure à la moyenne brute. Cela est dû au fait qu'elle est moins sensible aux valeurs aberrantes par rapport à la moyenne brute, et fournit une estimation plus robuste de la tendance centrale.
3. **Interprétation** : Si la distribution des valeurs de support est asymétrique ou contient des valeurs aberrantes, la moyenne tronquée fournit une mesure plus représentative de la tendance centrale par rapport à la moyenne brute.

Pour confirmer cette théorie, nous avons mené une expérience avec un échantillon de 100 données aléatoires, présentant une moyenne de 10 et un écart

type de 4. Nous avons également introduit des valeurs aberrantes (50, 60, 70) dans cet échantillon.



Comme indiqué dans le graphique, la moyenne tronquée est moins sensible aux valeurs aberrantes extrêmes que ne l'est la moyenne.

Apriori avec **min_sup** dynamique

Code initial

ci-dessous est l'algorithme a priori classique

```
Fonction Apriori(df, support_minimal)
    // Initialiser L1 comme l'ensemble des items fréquents de t
    // 1 qui satisfont le support minimal
```

```

L = [L1]
k = 2

Tant que L[k-2] n'est pas vide
    Ck = GénérerCandidats(L[k-2], k)
    Pour chaque transaction t dans df
        Ct = Sous-ensembles(t, Ck)
        Pour chaque candidat c dans Ct
            c.compte += 1
    Lk = {c ∈ Ck | c.compte / nombre total de transactions :
    L.ajouter(Lk)
    k += 1

Retourner Union(L)
Fin Fonction

```

Notre version proposée de l'algorithme Apriori ne prendra pas un `min_support` initial comme paramètre. À la place, elle utilisera un `percentile_de_trim` pour déterminer dynamiquement le support des itemsets. Ce paramètre, exprimé sous forme de valeur décimale comprise entre 0 et 1, ajuste la sensibilité de l'algorithme aux valeurs extrêmes dans la distribution des supports.

Des valeurs plus proches de 0 augmentent la sensibilité de l'algorithme aux valeurs extrêmes. À l'inverse, des valeurs plus proche de 0.5, augmentent le pourcentage de données exclues du calcul.

Code final

```

Fonction Apriori(df, percentile_de_trim)
    Initialiser frequent_item_sets comme un dictionnaire vide

    Pour niveau de 0 à nombre de colonnes dans df - 1
        candidats = CréerCandidats(df, niveau)

        Si candidats est vide

```

```
        Retourner frequent_item_sets

support_moyen_tronqué = ObtenirSupportMoyenTronqué(df, k)
frequent_item_sets[niveau] = ÉlaguerCandidats(candidats, support_moyen_tronqué)

Retourner frequent_item_sets
Fin Fonction
```

Tests

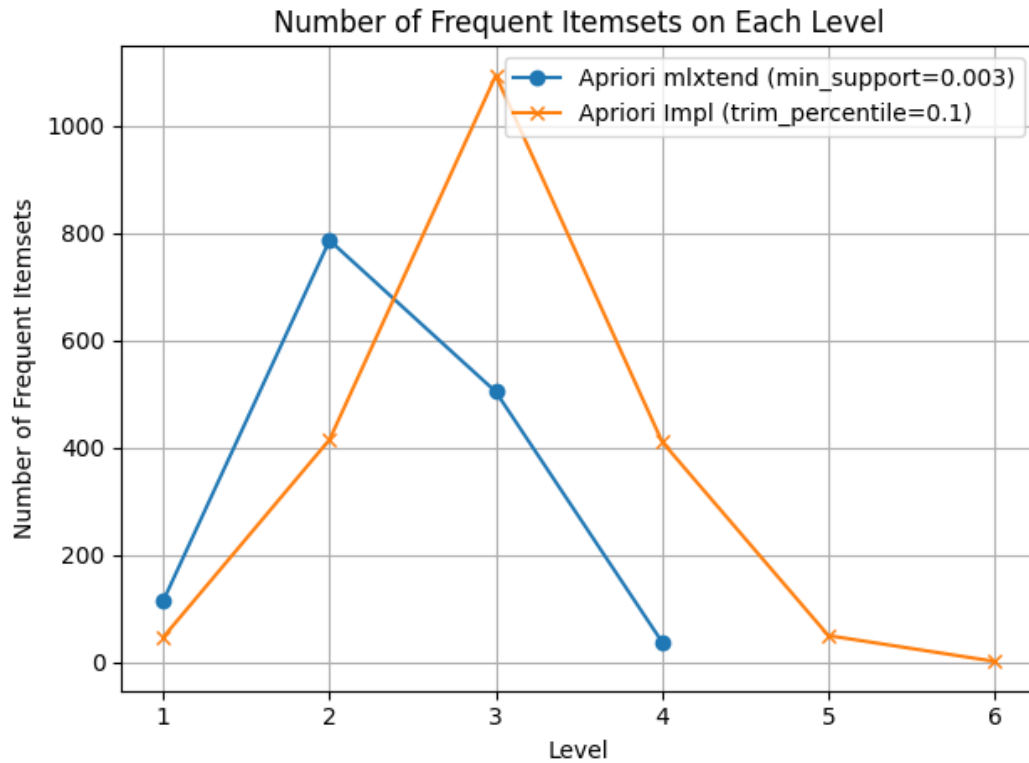
Dataset

Nous avons appliqué l'algorithme apriori sur l'ensemble de données Market Basket, où chaque ligne représente une transaction et chaque colonne représente un élément différent. Cet ensemble de données comprend 7 501 transactions et 119 éléments distincts.

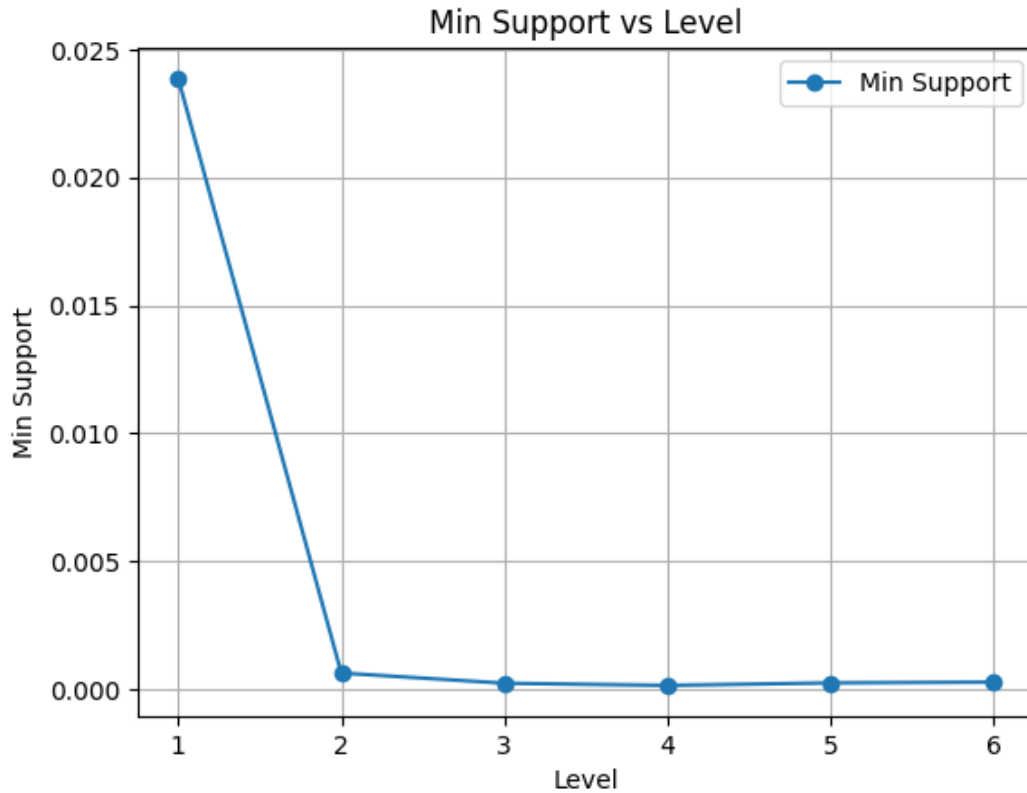
Préparation des données

Nous avons créé un script personnalisé qui transforme un ensemble de données de transaction en un ensemble binarisé.

cela nous aidera à accélérer les calculs du support minimum, car nous pouvons utiliser des opérations vectorielles efficaces pour compter le nombre de transactions dans lesquelles chaque ensemble d'articles apparaît.



Cette observation met en évidence le fait que l'approche de la moyenne tronquée dynamique a conduit à une meilleure capture des ensembles d'items fréquents par rapport à une approche utilisant un support minimal fixe. Cela suggère que la sensibilité accrue aux valeurs extrêmes, contrôlée par le paramètre de la moyenne tronquée, a permis de mieux adapter le seuil de support aux particularités des données.



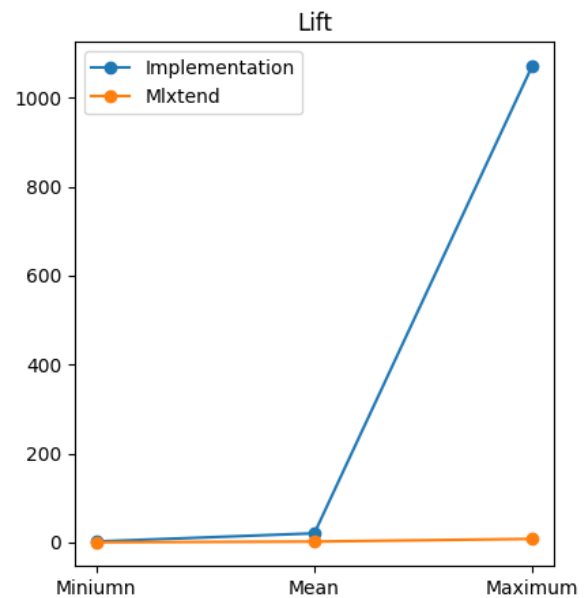
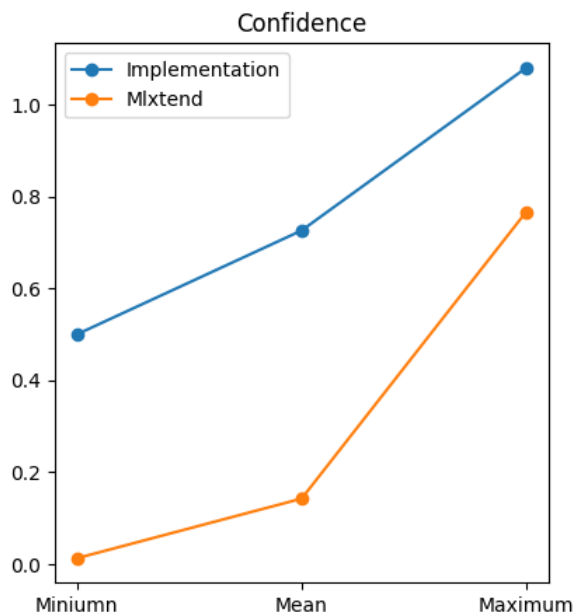
Comme nous pouvons le constater à partir de l'évolution de `min_sup`, nous constatons une forte baisse de `min_support`, ce qui signifie que l'utilisation d'un `min_sup` fixe a éliminé une grande partie de nos données, même si elles sont toujours pertinentes pour l'étude.

Règles d'association

L'amélioration de la génération fréquente d'ensembles d'éléments se propage également à l'extraction des règles d'association, car nous remarquons une confiance plus élevée et des métriques améliorées.

$$X \Rightarrow Y : \text{Confidence} = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$$

$$X \Rightarrow Y : \text{Lift} = \frac{\text{support}(X \cup Y)}{\text{support}(X) \times \text{support}(Y)}$$



Conclusion

dans ce projet, nous avons exploré une technique d'optimisation pour l'algorithme a priori, nous avons commencé par utiliser la moyenne, et avons remarqué qu'elle était sensible aux valeurs extrêmes, nous avons ensuite eu recours à une moyenne tronquée, ce qui nous a donné un min_support plus robuste pour différents niveaux, ce qui le rend adaptable aux pics de changements de fréquence à travers les niveaux.

nous avons également remarqué une amélioration notable dans l'extraction des règles d'association, où nous avons remarqué une augmentation de la confiance et une amélioration à tous les niveaux.